Documentație Tehnică: Sistemul LuggageFareCalculator

1. Introducere

Proiectul vizează dezvoltarea și testarea unei aplicații Java destinate calculării taxelor de bagaje pentru pasagerii aerieni. Sistemul ia în considerare clasa biletului (Economy/Business), statusul VIP al pasagerului, numărul de bagaje și greutatea fiecăruia pentru a determina tariful final.

2. Configurația Mediului de Lucru

2.1. Configurație Hardware

- **Procesor**: Intel Core i7
- **Memorie RAM**: 16 GB DDR4/LPDDR5
- **Stocare**: SSD NVMe (minim 250 GB)

2.2. Configurație Software

- **Sistem de Operare**: Windows 11
- **Limbaj de Programare**: Java (JDK 21)
- **Build Tool**: Apache Maven 4.0.0
- **IDE**: IntelliJ IDEA 2024

2.3. Versiuni Tool-uri de Testare

- **JUnit Jupiter (API & Engine)**: 5.10.2
- **PIT Mutation Testing (Plugin)**: 1.15.8
- **PIT JUnit 5 Plugin**: 1.2.1

3. Analiza Codului Sursă

Clasa principală, `LuggageFareCalculator`, gestionează logica de business prin următoarele componente cheie:

- **Constante**: Definirea limitelor de greutate (23.0 kg pentru Economy, 32.0 kg maxim absolut) și a taxelor (100.0 pentru bagaj extra, 50.0 per kg depășit).
- **Enumerarea TicketClass**: Definește categoriile `ECONOMY` și `BUSINESS`.
- **Metoda addLuggage**: Validează numărul maxim de bagaje (5) și greutatea fiecăruia înainte de a le adăuga în listă.

- **Metoda calculateTotalFee**: Calculează tariful total prin iterarea listei de bagaje și aplicarea regulilor de gratuitate și supratarxare.

```
// Fragment din logica de calcul a taxelor
for (int i = 0; i < luggageWeights.size(); i++) {
  double weight = luggageWeights.get(i);
  boolean isExtraBag = i >= freeBagsAllowed;

  if (isExtraBag && !isVip) {
    totalFee += EXTRA_BAG_FEE;
  }
  if (weight > allowedWeight && !isVip) {
    totalFee += (weight - allowedWeight) * OVERWEIGHT_FEE_PER_KG;
  }
}
```

4. Diagrame UML și Analiza Fluxului

4.1. Diagrama de Clase

Structura sistemului și relațiile dintre clasele de test și clasa de bază sunt ilustrate mai jos:

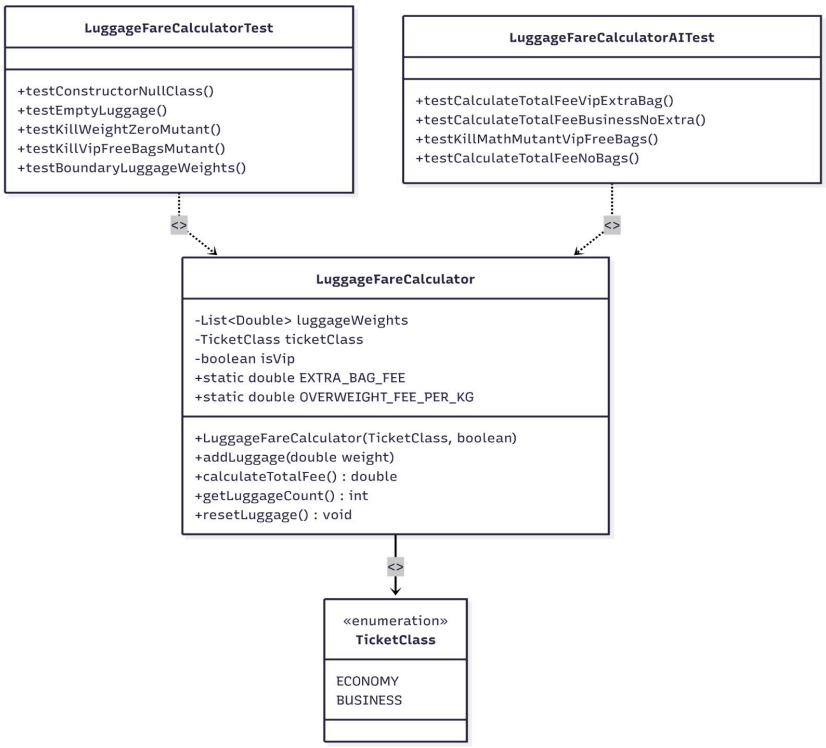

Figura 1: Diagrama UML a claselor sistemului.

4.2. Graful Fluxului de Control (CFG)

Analiza complexității metodei calculateTotalFee a fost realizată prin identificarea nodurilor decizionale și a căilor independente:

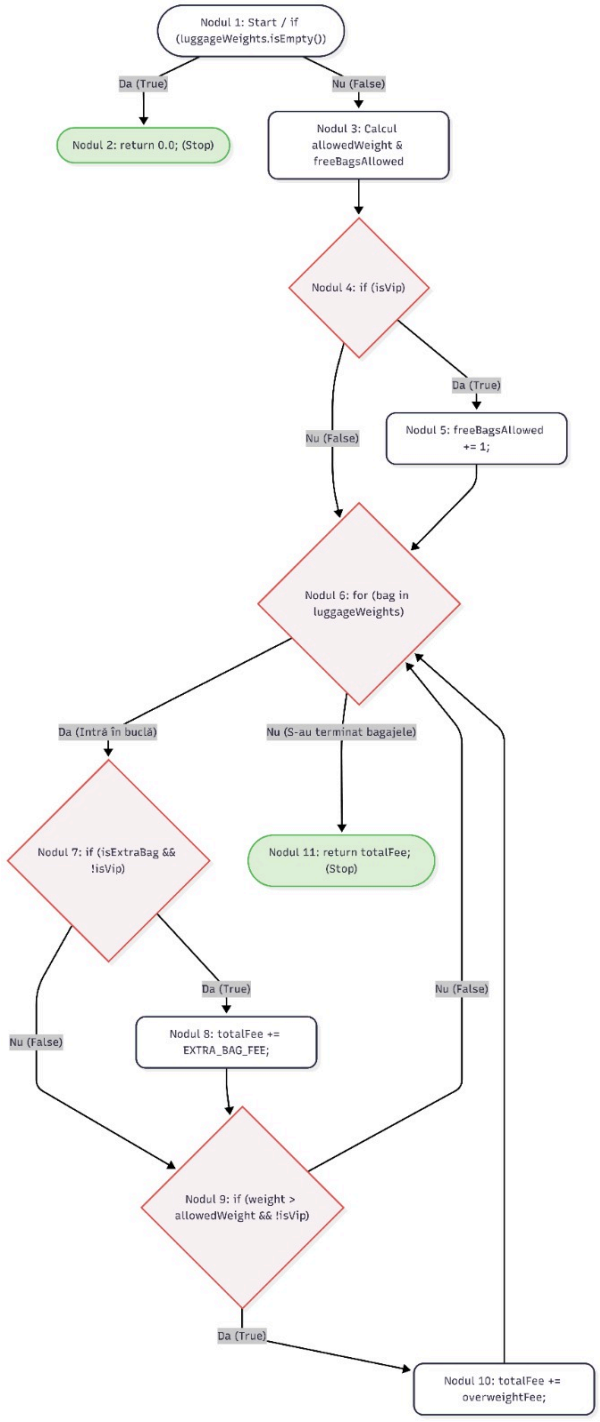

Figura 2: Graful fluxului de control (CFG) pentru calculul taxelor.

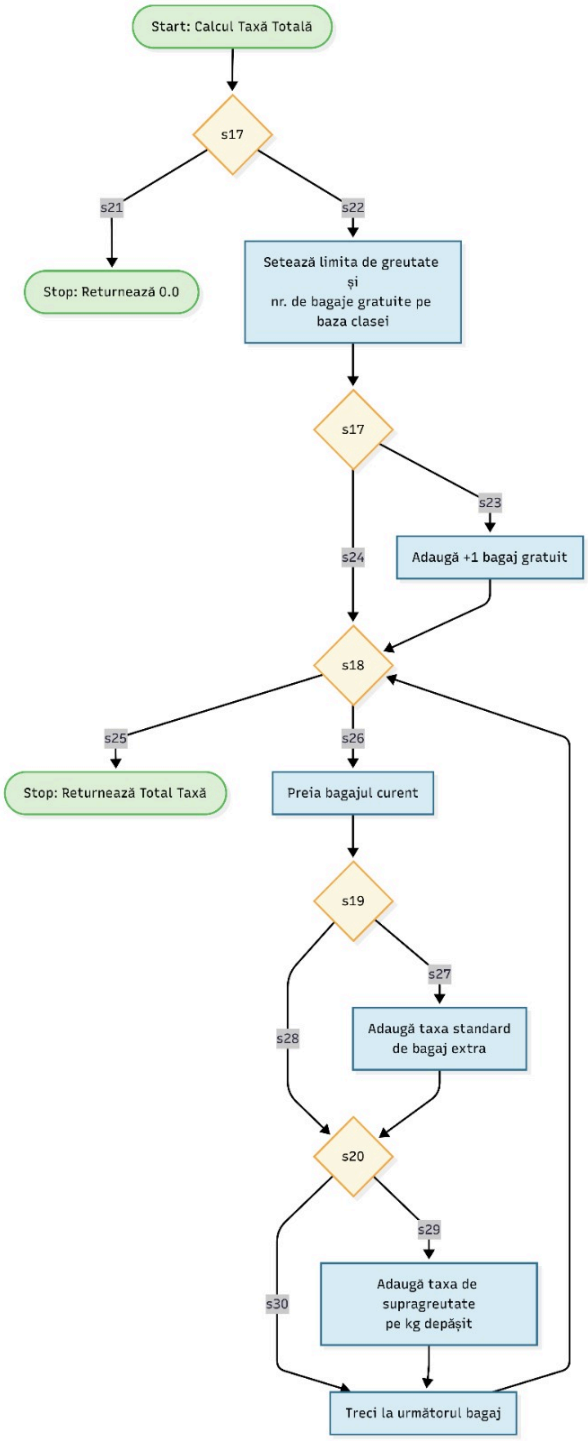

Figura 3: Diagramă de flux detaliată a procesului de calcul.

5. Strategii de Testare

Procesul de verificare a inclus patru strategii majore:

1. **Analiza Valorilor de Frontieră (BVA)**: Validarea limitelor de greutate (0.0, 0.01, 32.0, 32.01 kg).
2. **Partiționarea în Clase de Echivalență (EP)**: Verificarea constructorului cu parametri null.
3. **Complexitatea Ciclomatică (McCabe)**: Acoperirea tuturor ramurilor decizionale identificate în CFG.
4. **Mutation Testing**: Utilizarea PIT pentru a genera mutanți la nivel de operatori și limite relaționale.

6. Comparație Rezultate: Testare Manuală vs. AI

Indicator PIT	**Suita Manuală (Ours)**	**Suita AI (AITest)**	**Interpretare**
Line Coverage	94% (31/33 linii)	**100% (33/33 linii)**	AI-ul a inclus teste pentru getteri și resetare.
Mutation Coverage	84% (26/31 mutanți)	84% (26/31 mutanți)	Ambele suite au ratat aceiași mutanți logici.
Test Strength	**93%**	84%	Testele manuale sunt mai dense pe logica acoperită.

6.1. Capturi de Ecran Rulare PIT

Rezultatele rulării automate a testelor reflectă acoperirea și eficiența acestora:

- **Suita Manuală**: A omis testarea metodelor getTicketClass și isVip, rezultând în 94% acoperire.

- **Suita AI**: A atins acoperire completă prin metoda testUtilityMethodsAndReset, demonstrând o capacitate superioară de a identifica zonele "uitate" de testerul uman.

7. Interpretări și Concluzii

Analiza comparativă relevă faptul că instrumentele de AI sunt extrem de utile pentru generarea rapidă a codului de tip "boilerplate" și atingerea unei acoperiri exhaustive a liniilor de cod (100%). Totuși, calitatea logicii de testare (Mutation Coverage) rămâne similară, ambele abordări eșuând în a ucide mutanții subtili din condițiile logice compuse (ex: liniile 62 și 68). Se recomandă utilizarea AI ca punct de plecare, urmată de o rafinare manuală a cazurilor complexe.

8. Referințe Bibliografice

1. **Ammann, P. & Offutt, J.**, *Introduction to Software Testing*, Cambridge University Press, 2016 (Citat pentru EP și BVA).

2. **JUnit 5 User Guide**, v.5.10.2, https://junit.org/junit5/docs/current/user-guide/.

3. **PIT Mutation Testing Documentation**, https://pitest.org/.

4. **Cod Sursă Proiect**: LuggageFareCalculator.java, LuggageFareCalculatorTest.java, LuggageFareCalculatorAITest.java.